Architectural Decision-Making & Trade-Offs

Mastering Software Architecture Patterns and System Design

Rahul Rajat Singh

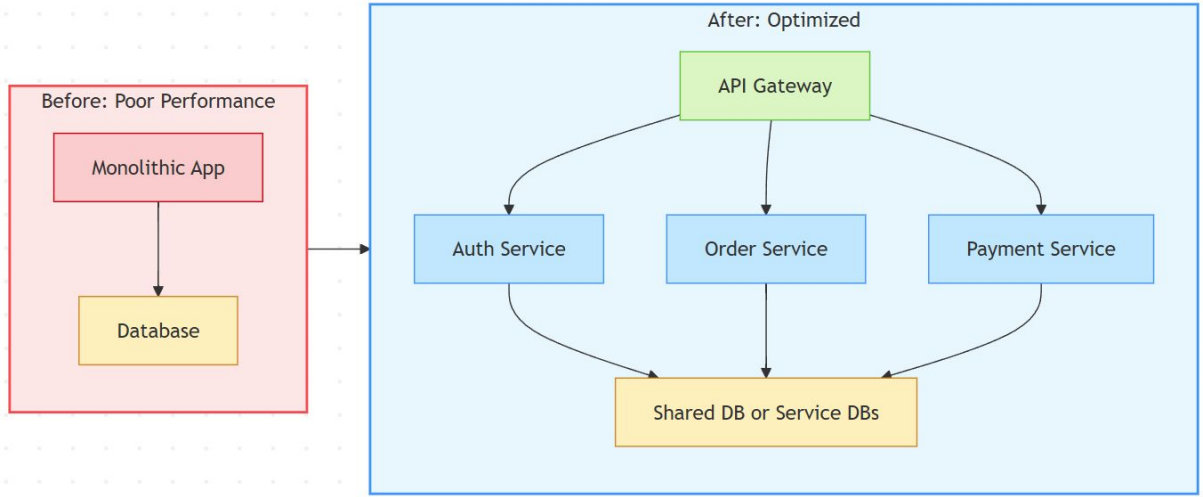
Choosing the Right Architecture Pattern

Architectural Decision-Making & Trade-Offs

Mastering Software Architecture Patterns and System Design

Why Architecture Patterns Matter

- Drive scalability, maintainability, and performance
- Impact development speed & team coordination
- Patterns are trade-offs, not silver bullets
- Wrong choice = long-term technical debt



Types of Architecture Patterns

- Monolithic
- Microservices
- Event-Driven
- Layered / N-Tier
- Serverless / Function-based

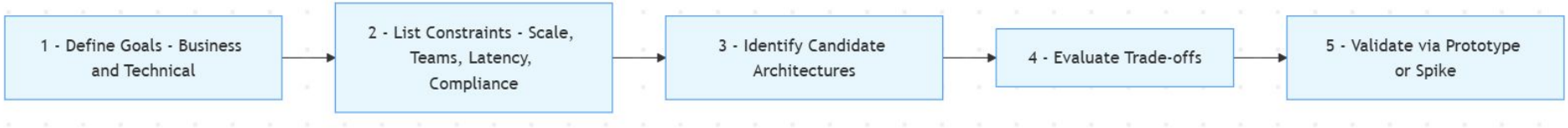


The Core Criteria for Choosing an Architecture

- System complexity and domain boundaries
- Scale: current load vs. future growth
- Team structure and size
- Time-to-market pressure
- Regulatory/security constraints

Step-by-Step Architecture Selection Process

- Define business & technical goals
- List constraints (scale, teams, latency, compliance)
- Identify candidate architectures
- Evaluate trade-offs
- Validate via spike or prototype



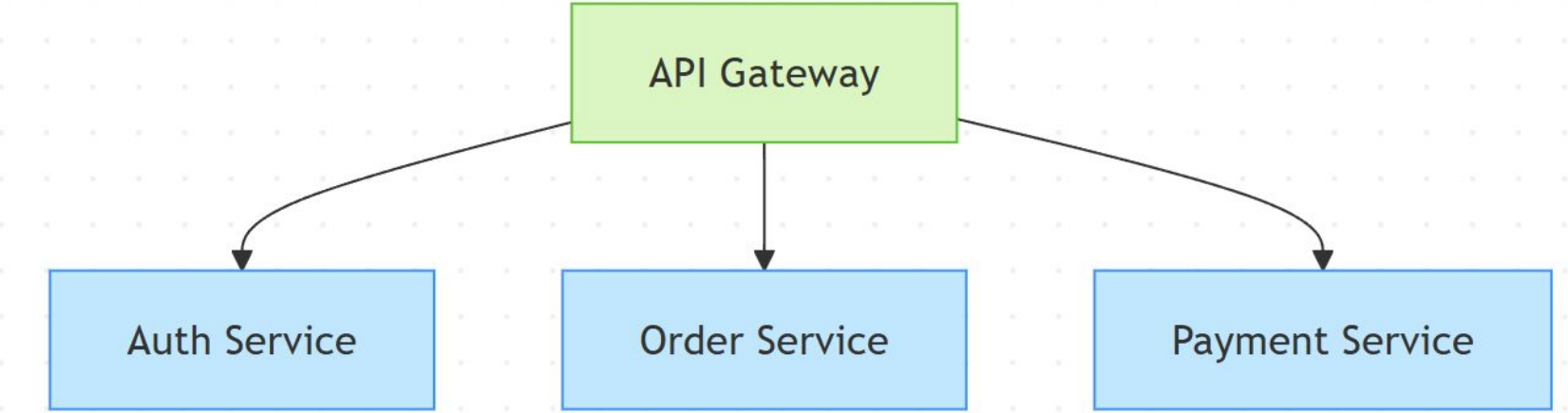
Monolithic Architecture – When It's a Good Fit

- Simple applications, small teams
- Fast iteration & deployment
- Centralized logging and debugging
- Can be a valid MVP approach



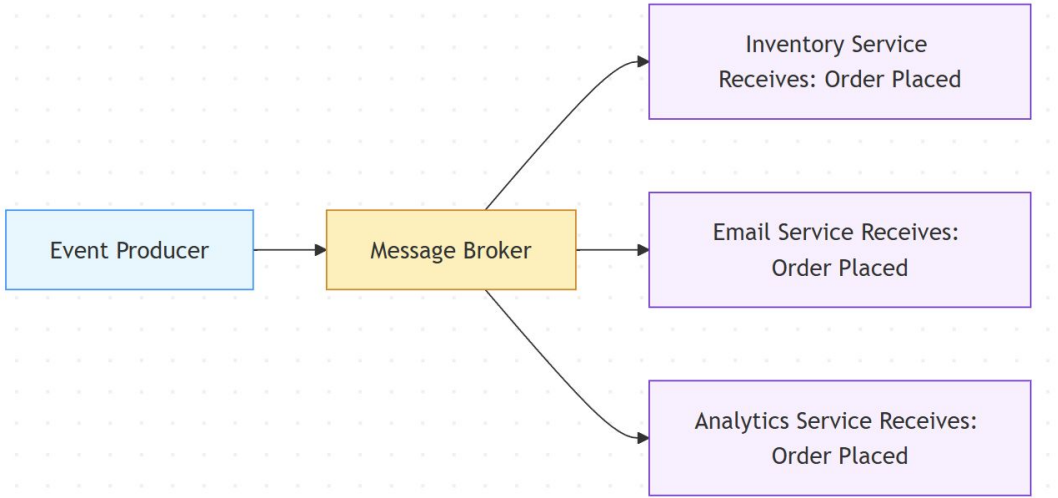
Microservices Architecture – Strengths & Constraints

- Great for large-scale systems with clear domains
- Independent deployability and scaling
- Needs strong DevOps and observability
- Increased complexity and latency risks



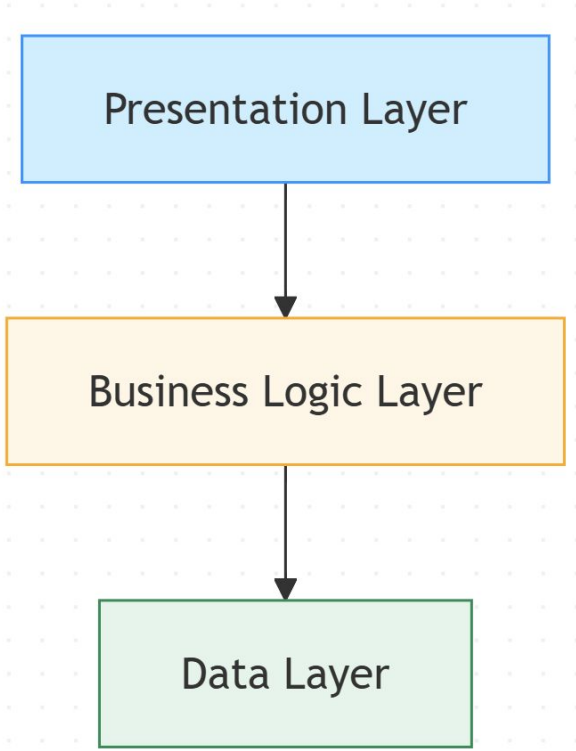
Event-Driven Architecture – Use Cases

- Asynchronous, loosely coupled communication
- Ideal for real-time systems, logs, and analytics
- Enables decoupling and flexibility
- Challenges: ordering, retries, eventual consistency



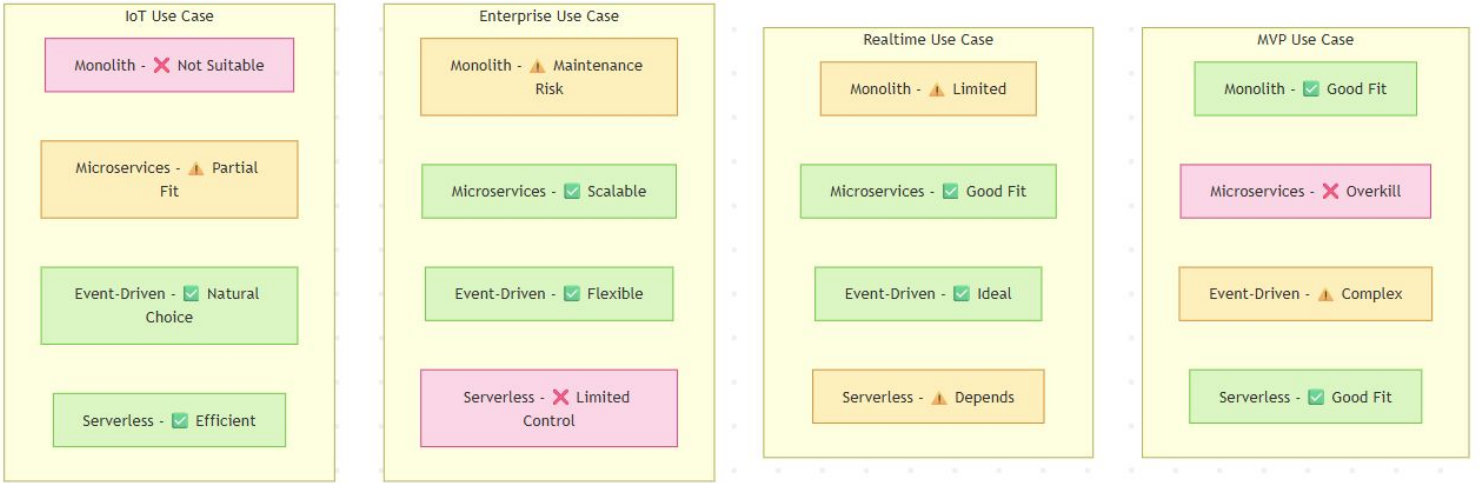
Layered / N-Tier Architecture – Still Relevant

- Common in enterprise, especially with legacy systems
- Separation of concerns: UI, business logic, data
- Supports modular design, easy onboarding
- Can get rigid or layered too deeply



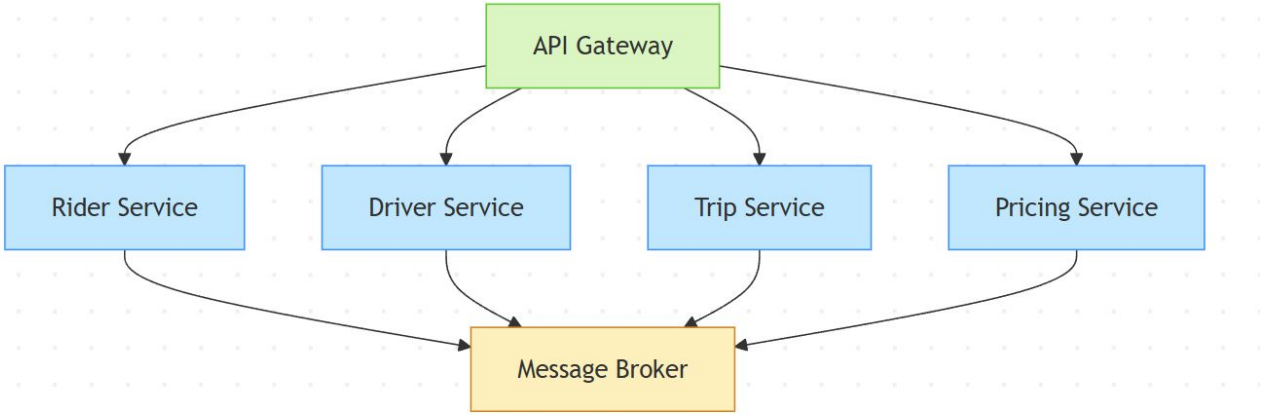
Architecture Fit Matrix (Use Case vs. Pattern)

- Map use cases to ideal architectures
- E.g., MVP → Monolith, Realtime → Event-Driven
- One size does not fit all
- Hybrid models are often needed



Example: Choosing for a Ride-Hailing App

- Business goals: real-time updates, high availability
- Team: 6 backend devs, experienced with microservices
- Constraints: Latency, consistency, scaling
- Chosen pattern: Microservices + Event-Driven hybrid



Summary - Choosing the Right Architecture Pattern

- Start with understanding constraints and goals
- Evaluate patterns as trade-offs, not recipes
- Simpler is often better for early-stage systems
- Revisit architecture as systems evolve
- What's next:
  - Trade-Off Analysis

Trade-Off Analysis

Architectural Decision-Making & Trade-Offs

Mastering Software Architecture Patterns and System Design

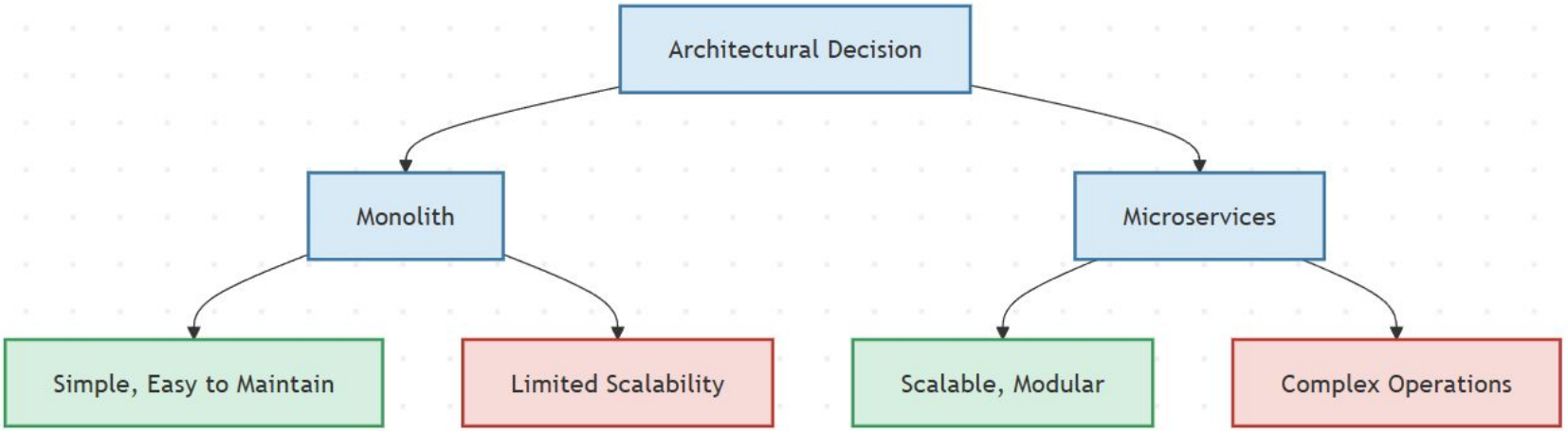
Introduction to Trade-Offs in System Design

- Every architectural decision has consequences
- Trade-offs arise due to conflicting goals or constraints
- No "perfect" solution—only best-fit for a context
- Awareness of trade-offs = smarter design decisions



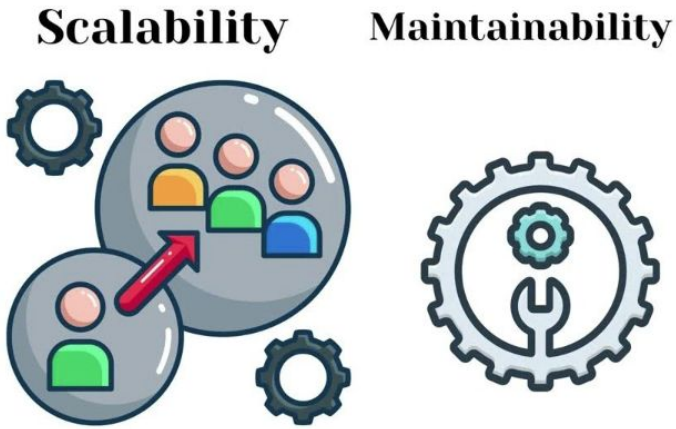
Why Trade-Off Analysis Matters

- Enables informed and intentional decision-making
- Avoids over-engineering or premature optimization
- Guides architecture reviews and design documents
- Clarifies stakeholder communication (product vs tech teams)



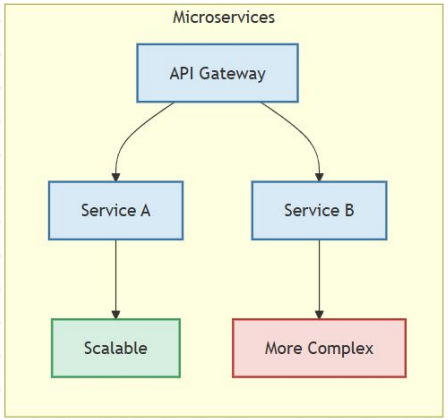
Scalability vs Maintainability

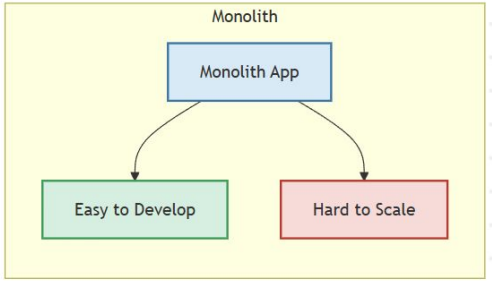
- Scalability: Can the system handle growth in users/data?
- Maintainability: Can the system be easily modified or debugged?
- Highly scalable systems often involve complexity (microservices, distributed state)
- Simpler, monolithic designs often win on maintainability (at small scale)



Real-World Example: Monolith vs Microservices

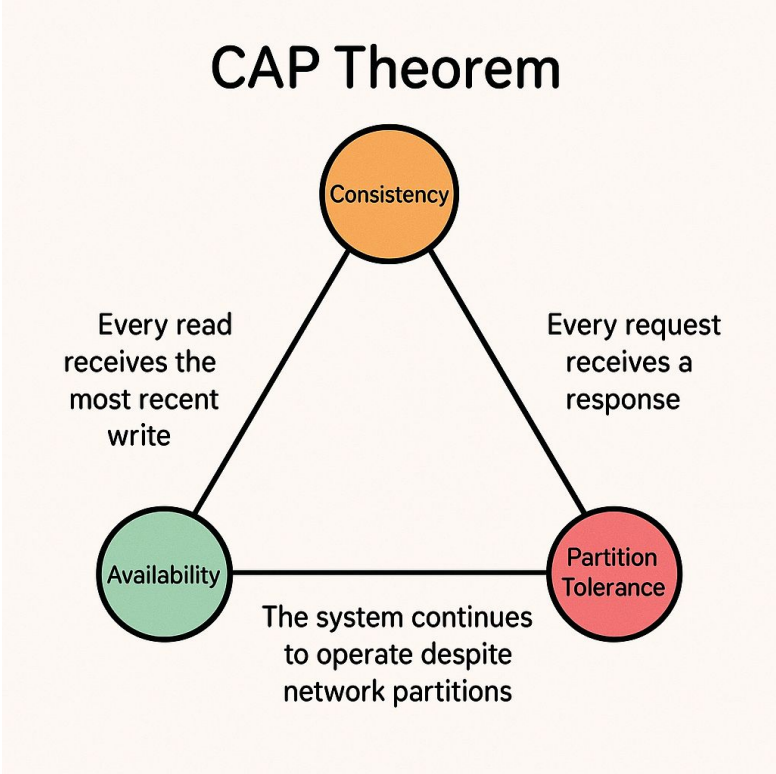
- Monolith: Easy to debug, test, and deploy—but limited in scalability
- Microservices: Scalable and modular—but require orchestration, observability
- Choose monoliths for early startups, microservices for mature systems





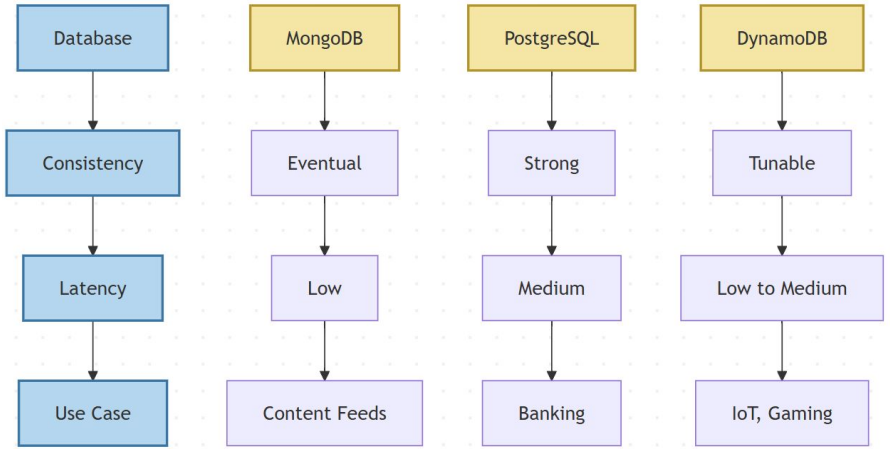
Latency vs Consistency

- Consistency: Ensuring data correctness across systems
- Latency: Time taken to respond to a request
- Strong consistency often increases latency (e.g., distributed locks)
- Eventual consistency reduces latency but introduces complexity



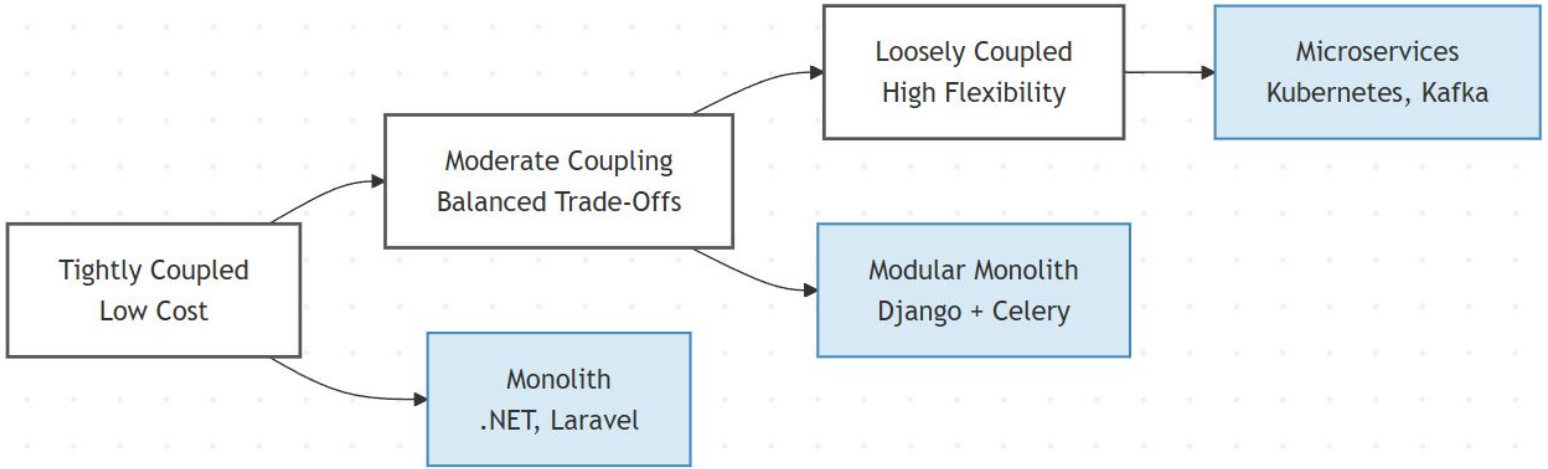
Use Case: Distributed Databases

- Examples: Cassandra, DynamoDB, MongoDB
- Choose eventual consistency for high availability and speed
- Strong consistency (e.g., SQL-based systems) suitable for critical systems like banking
- Trade-off shifts based on use case



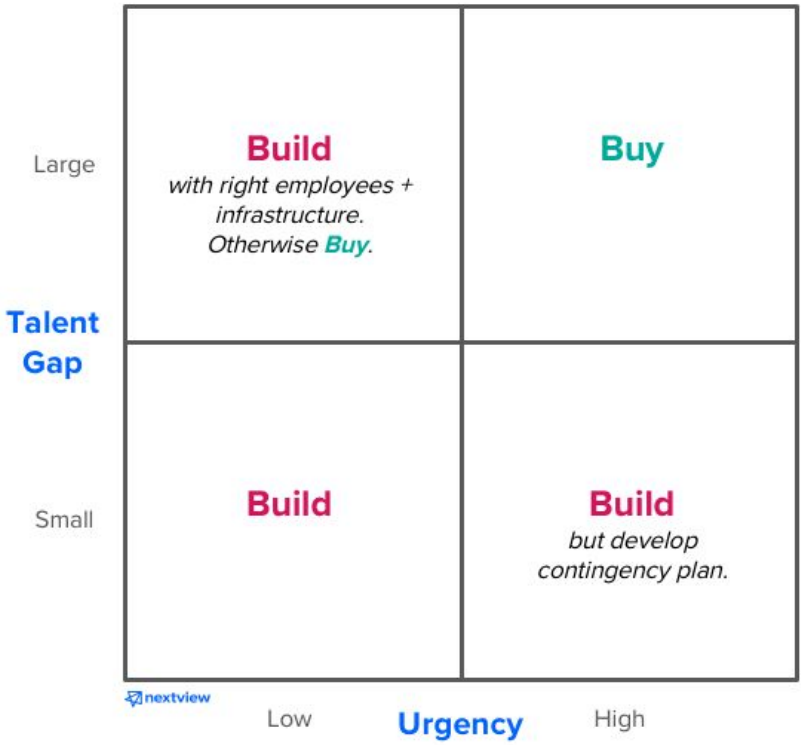
Cost vs Flexibility

- Flexibility = ability to adapt system to new requirements
- High flexibility often requires modular, loosely coupled systems
- These systems can be costly in terms of development, infrastructure
- Tightly coupled systems are cheaper but harder to adapt



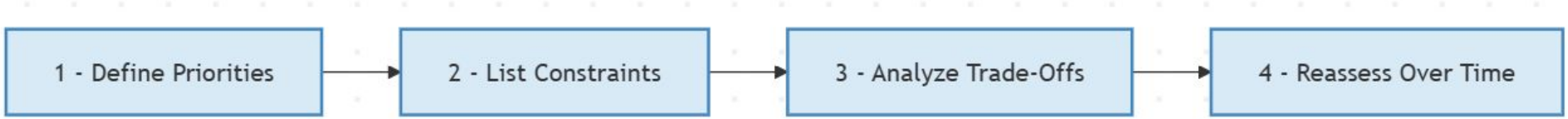
Scenario: Build vs Buy Decision

- Buying saves time/cost short-term but limits control/flexibility
- Building offers customization but consumes engineering time
- Example: Authentication as a Service (e.g., Auth0 vs in-house)
- Balance cost of ownership with adaptability needs



Framework for Evaluating Trade-Offs

- Identify key priorities: What matters most now?
- Clarify constraints: Time, team, traffic, tech
- Evaluate options across dimensions
- Revisit decisions as the system evolves



Summary: Key Trade-Offs

- Scalability vs Maintainability: Growth vs simplicity
- Latency vs Consistency: Speed vs correctness
- Cost vs Flexibility: Budget vs adaptability
- No universal answer—context is everything
- What’s next:
  - Pattern Selection Frameworks

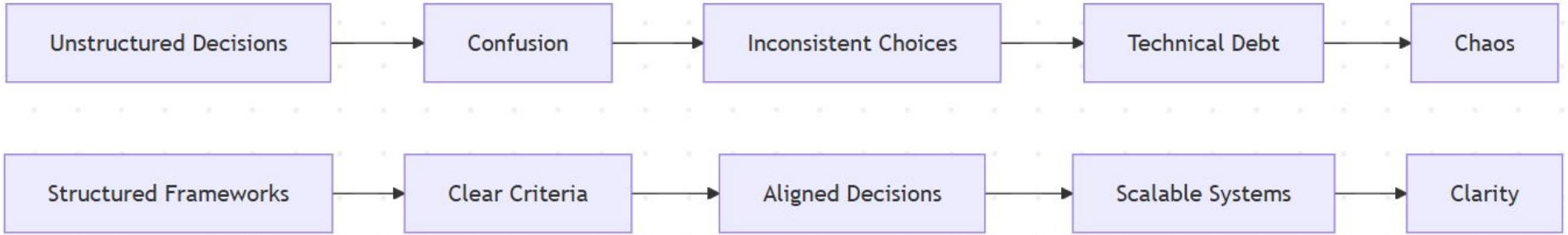
Pattern Selection Frameworks

Architectural Decision-Making & Trade-Offs

Mastering Software Architecture Patterns and System Design

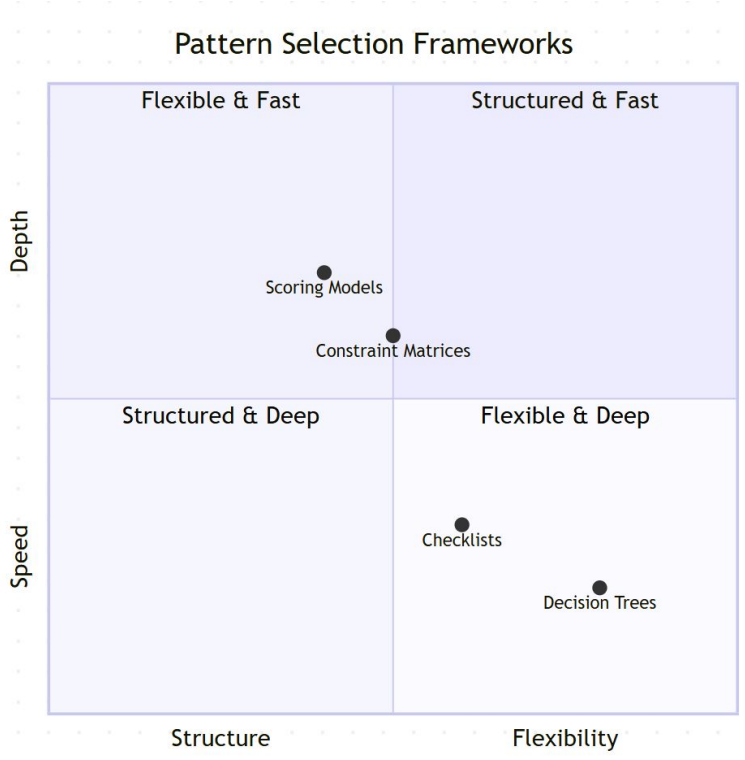
The Need for Selection Frameworks

- Architecture involves multiple forces & constraints
- Ad-hoc decisions = high risk, harder to justify
- Frameworks reduce bias, improve repeatability
- Enables shared understanding across teams



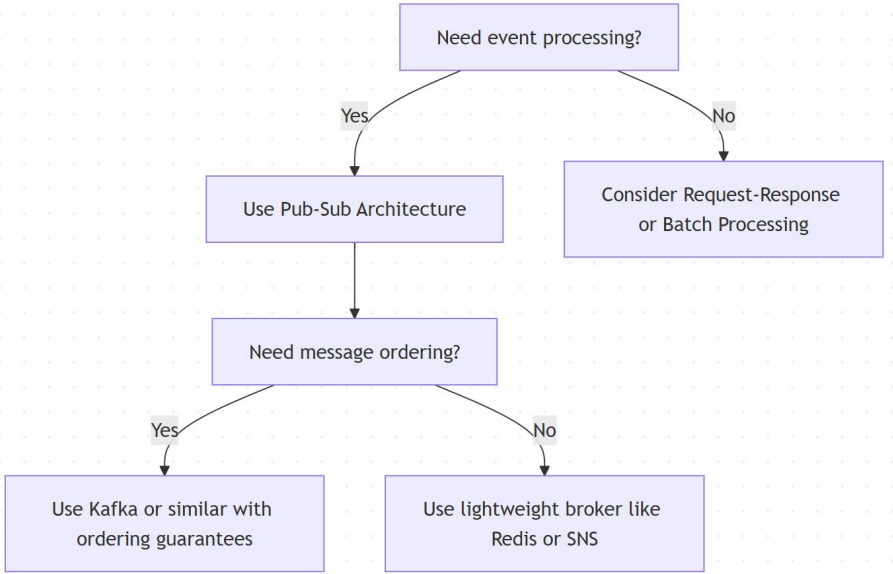
Types of Pattern Selection Frameworks

- 🧭 Decision Trees — Rule-based branching logic
- ✅ Checklists — Ensure coverage of key factors
- 📊 Scoring Models — Structured comparisons
- 📐 Constraint Matrices — Visualize feasibility vs goals



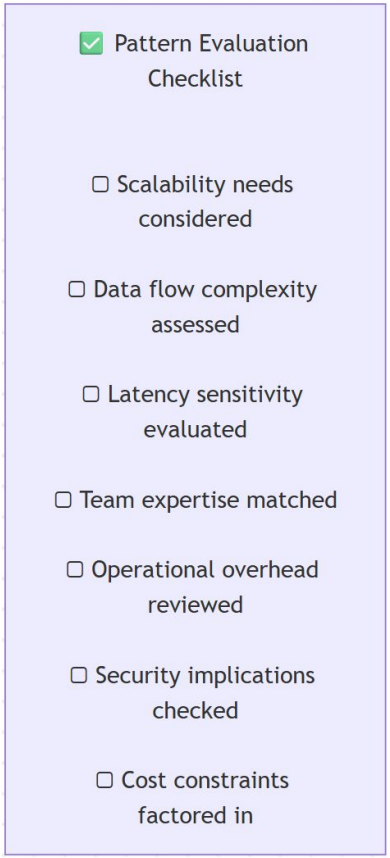
Decision Trees: Rule-Based Navigation

- Used for standard or recurring decision paths
- Encodes experience in “if → then” logic
- Works well for small, well-scoped domains
- Becomes rigid at scale or with fuzzier input



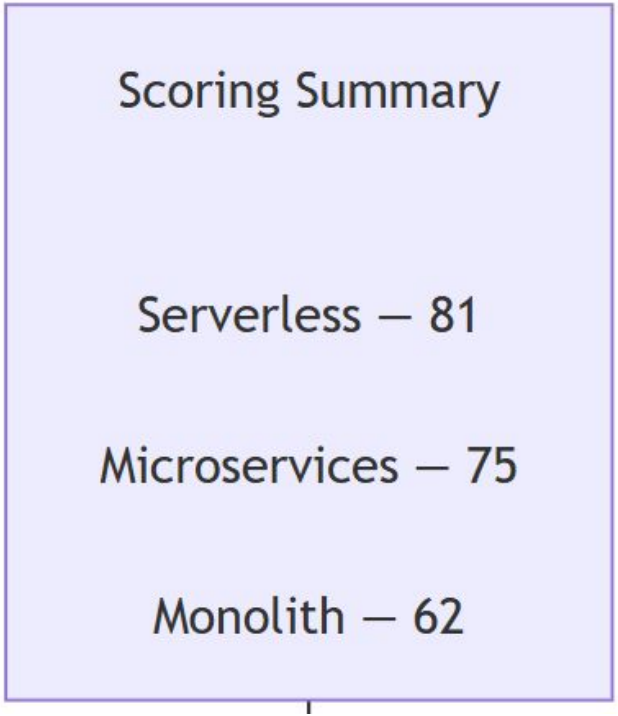
Checklists: Design Sanity Safeguard

- Focus on coverage, not scoring
- Use during pattern reviews or trade-off discussions
- Easy to create, customize, and scale across teams
- Danger: becomes a tick-box routine without critical thinking



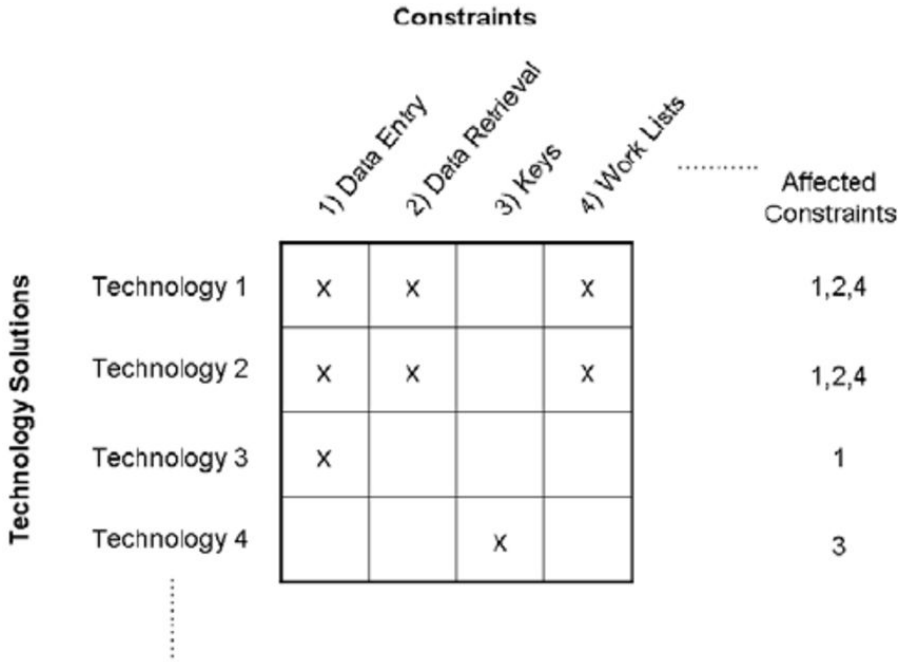
Scoring Models: Structured Comparison

- Assign weights to decision factors (e.g., latency 30%)
- Score each pattern on each factor
- Sum weighted scores → rank options
- Good for aligning large groups



Constraint Matrix: Visual Feasibility Grid

- Plot patterns against critical constraints
- Use for fast elimination or alignment mapping
- Great for early-phase architecture discussions
- Visual → easy to present & explain



When to Use Which Framework

- Use Decision Trees for: automation, playbooks
- Use Checklists for: reviews, onboarding
- Use Scoring Models for: strategic comparisons
- Use Constraint Matrices for: stakeholder alignment

Decision Matrix

Fast decisions:
✓ Decision Tree

Reviews:
✓ Checklist

Team alignment:
✓ Checklist
✓ Scoring Model

Complex trade-offs:
✓ Scoring Model

Early elimination:
✓ Constraint Matrix

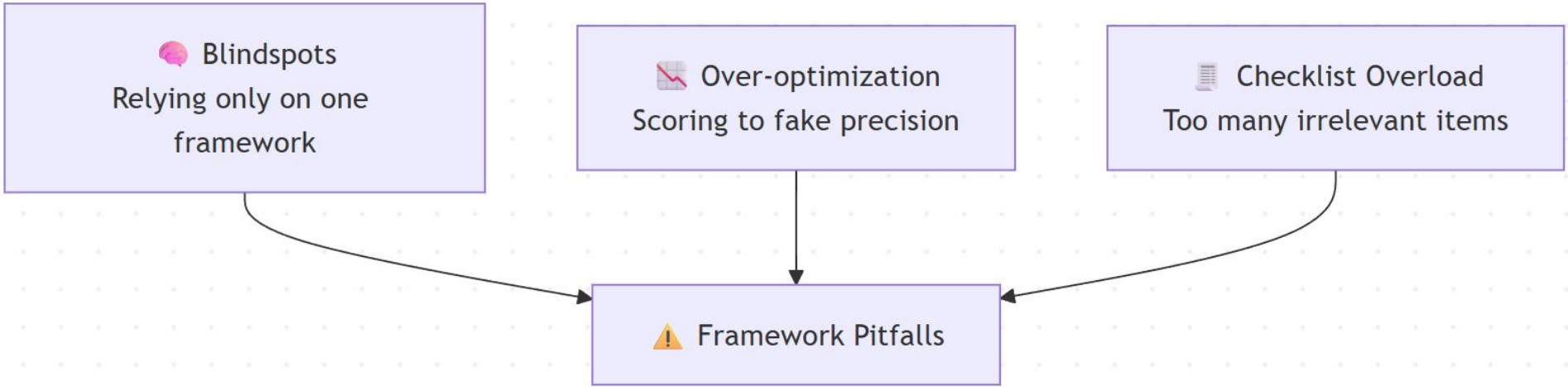
Creating Your Own Frameworks

- Start from your domain’s repeated decisions
- Identify common constraints and priorities
- Customize scoring weights or checklist items
- Keep it lightweight and iteratively improve



Common Pitfalls

- Overfitting: designing for yesterday's problems
- Ignoring people: team skills matter as much as tech
- Overscoring: false precision, illusion of objectivity
- Checklist fatigue: more items ≠ better design



Summary - Pattern Selection Frameworks

- Frameworks = structured thinking tools, not answers
- Use different types for different stages of decision
- Keep them lightweight, adaptable, team-aligned
- Update regularly as your systems and team evolve
- What's next:
  - Architecture Anti-Patterns & Failure Stories

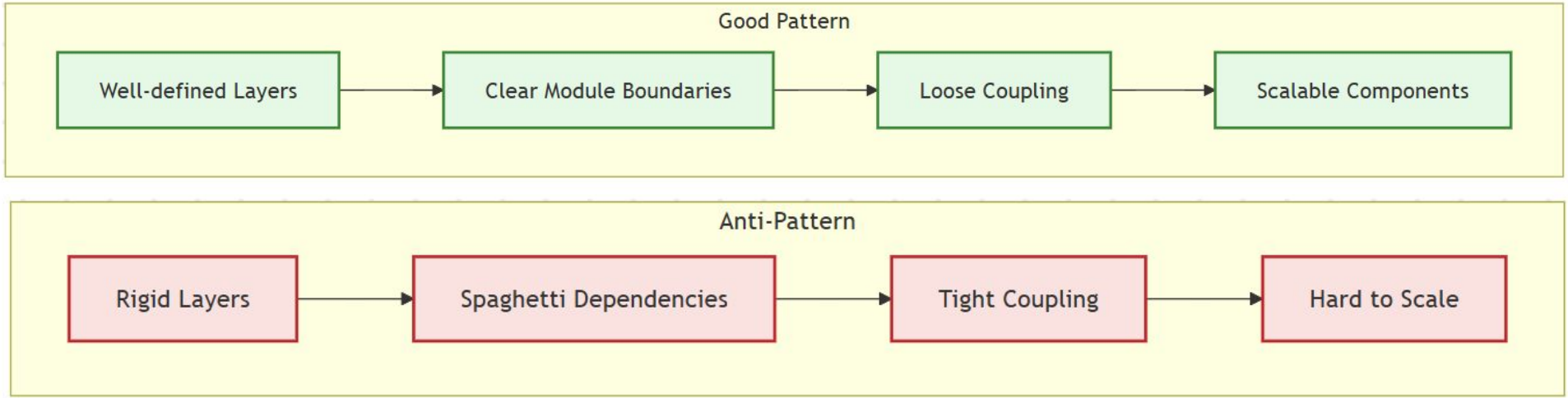
Architecture Anti-Patterns & Failure Stories

Architectural Decision-Making & Trade-Offs

Mastering Software Architecture Patterns and System Design

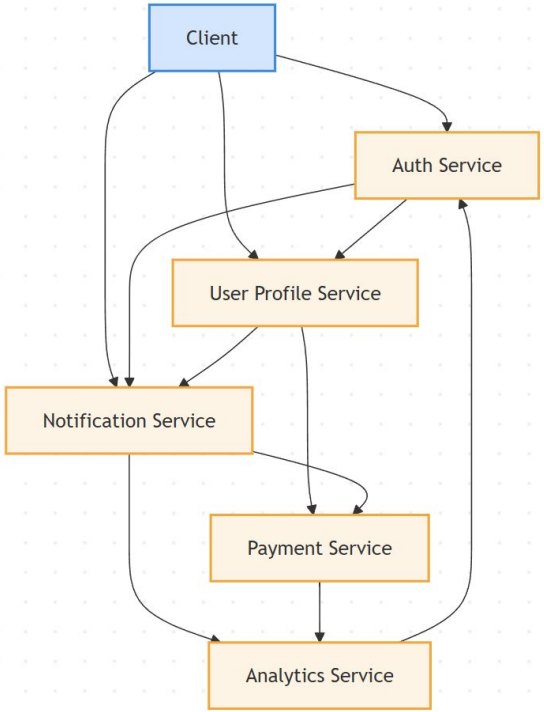
What Are Architecture Anti-Patterns?

- Repeated poor solutions to recurring design problems.
- Often arise from over-engineering or ignoring system constraints.
- Can lead to technical debt, poor performance, and team burnout.
- Early detection = faster recovery.



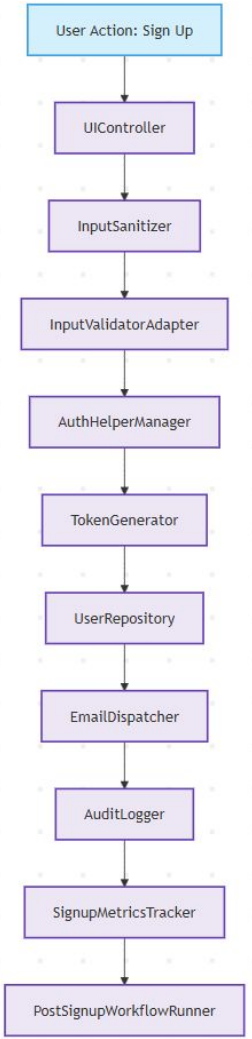
Chatty Microservices

- Too many small services making excessive calls to each other.
- High latency, brittle dependencies, and noisy networks.
- Fails the “single business capability per service” principle.
- Anti-pattern of over-decomposition.



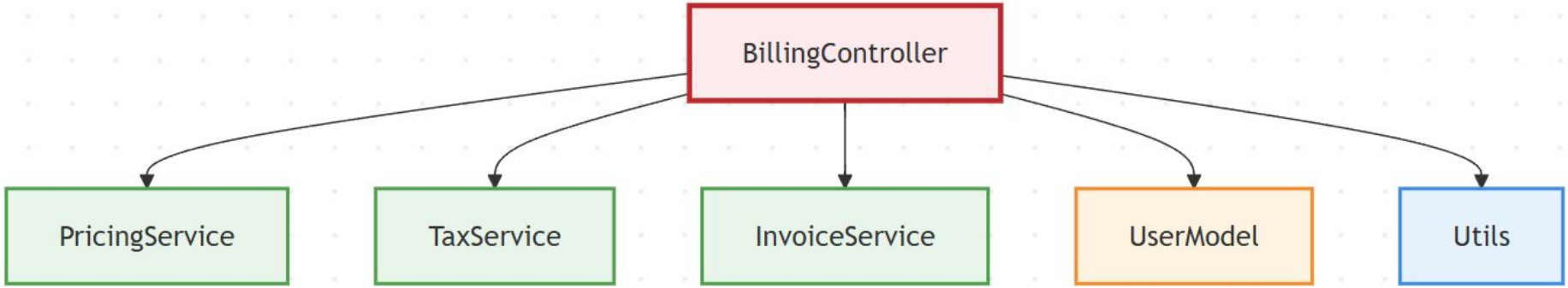
Over-Modularization

- Excessive splitting of responsibilities into tiny modules or layers.
- High overhead in coordination, integration, and cognitive load.
- Slows down onboarding and changes.
- Symptoms: Circular dependencies, unnecessary abstraction layers.



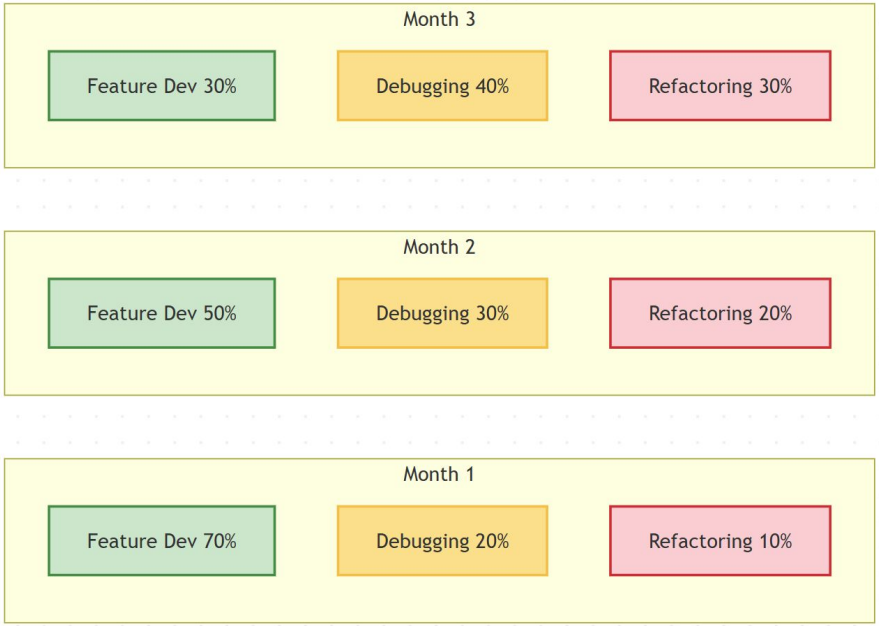
God Classes (aka "The Everything Controller")

- One class/module that knows and does too much.
- Centralization = coupling, testing nightmares, zero reusability.
- Violates single responsibility principle (SRP).
- Tends to grow unchecked over time.



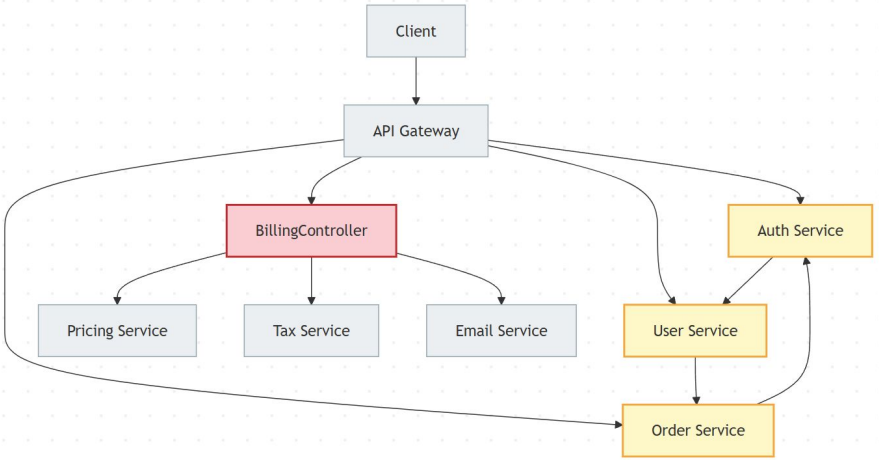
The Hidden Cost of Anti-Patterns

- Latency and downtime during peak hours.
- Team frustration and burnout.
- Delays in feature rollouts.
- High cost of rewrites or re-architecture.



Avoiding the Trap – Red Flags to Watch For

- Too many small services/modules doing too little.
- Growing LOC or scope in a central controller.
- Teams needing 5 meetings to align on small changes.
- Fragile systems that break under small outages.



Summary - Architecture Anti-Patterns & Failure Stories

- Anti-patterns emerge from good intentions gone unchecked.
- Watch out for chatty services, over-modularity, god classes.
- Real-world failures teach more than theoretical best practices.
- Design with cohesion, context, and change in mind.
- What's next:
  - Influence of Domain-Driven Design

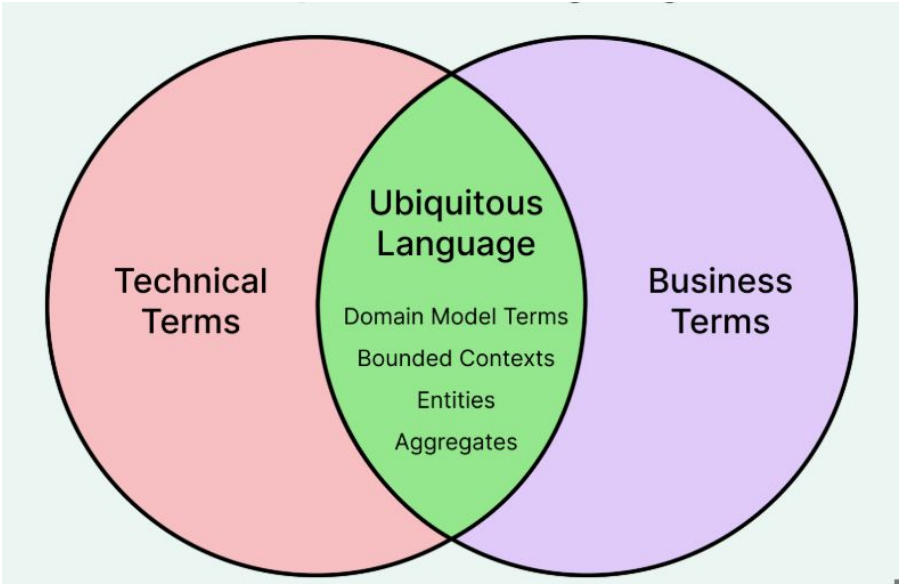
Influence of Domain-Driven Design

Architectural Decision-Making & Trade-Offs

Mastering Software Architecture Patterns and System Design

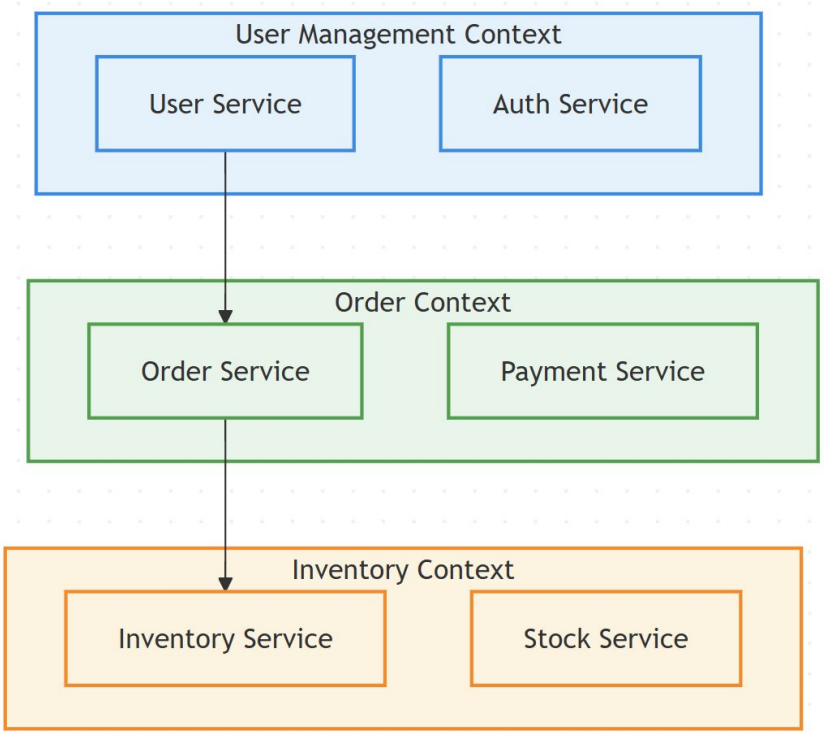
DDD Connects Business & Architecture

- Aligns software structure with business domains
- Creates shared language for teams & stakeholders
- Drives architectural boundaries rooted in domain logic
- Reduces miscommunication and design guesswork



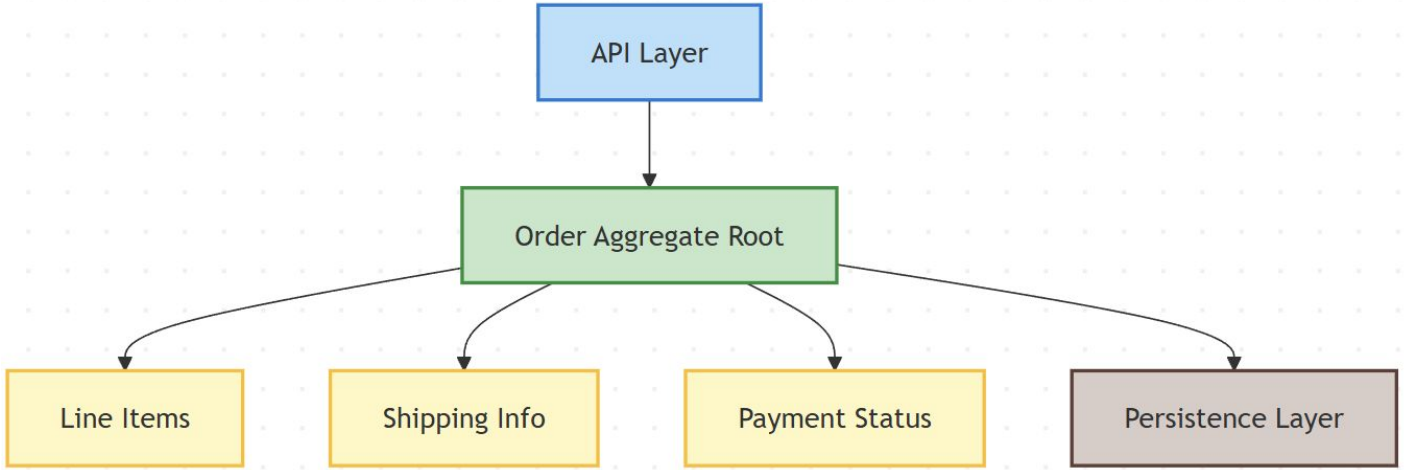
Bounded Contexts – Defining Clear Architectural Boundaries

- Each bounded context encapsulates a domain model
- Enforces internal consistency, enables independent evolution
- Guides service boundaries in microservices or modules in monoliths
- Prevents ambiguity by isolating terms and rules



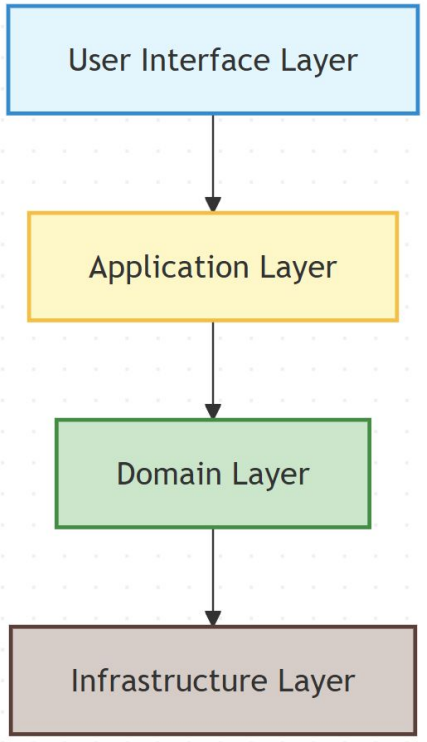
Aggregates Shape Data & Transaction Boundaries

- Aggregates enforce invariants inside boundaries
- Define transactional scope to keep systems responsive
- Reduce complexity by grouping related entities
- Guide database schema design and API contracts



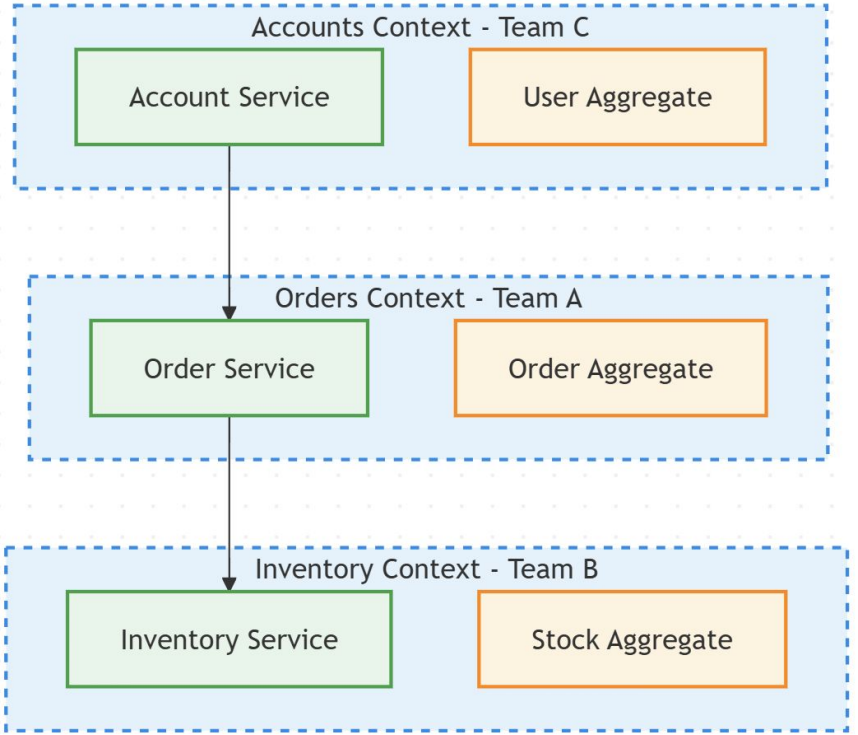
Layered Architecture Through a DDD Lens

- Domain layer contains pure business logic
- Application layer coordinates workflows without business rules
- Infrastructure layer handles persistence, messaging, and UI
- Promotes separation of concerns, testability, and flexibility



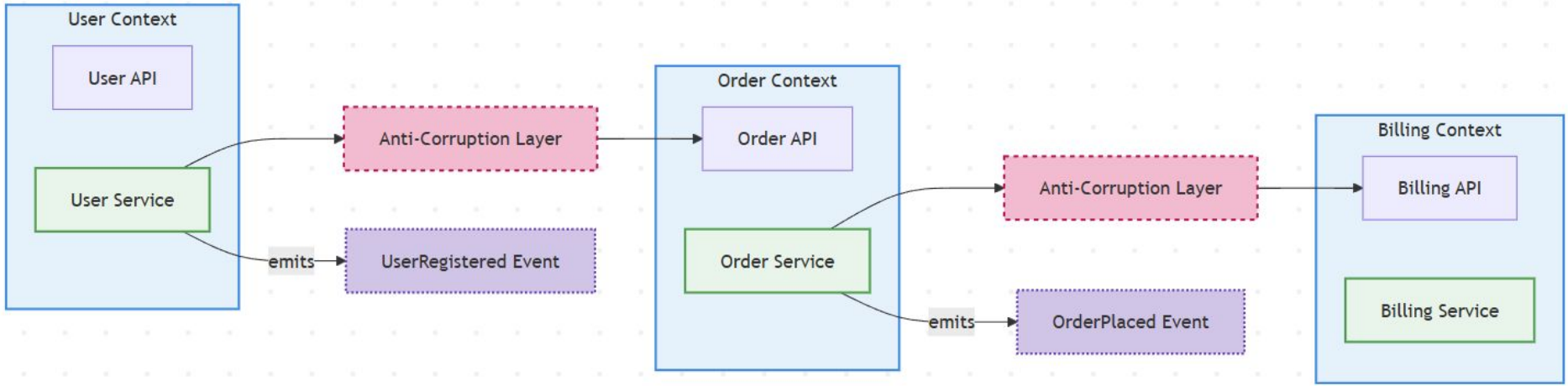
DDD Drives Modular & Scalable Architecture

- Bounded contexts naturally define modules or microservices
- Aggregates prevent cross-module data corruption
- Encourages teams to own vertical slices aligned with business functions
- Supports independent development, deployment, and scaling



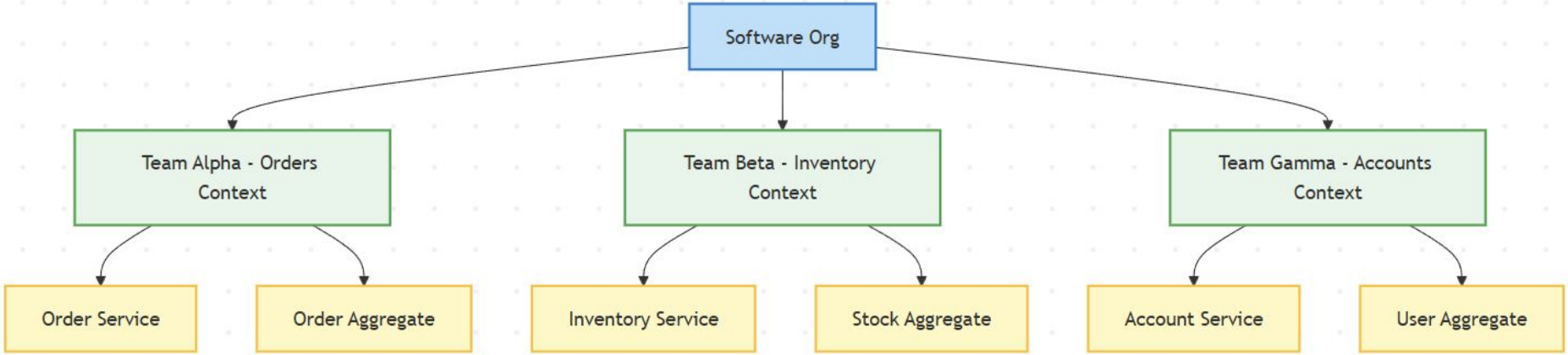
DDD Enables Strategic Integration Patterns

- Defines clear contracts and anti-corruption layers between contexts
- Supports synchronous and asynchronous communication
- Facilitates eventual consistency where appropriate
- Encourages using domain events for loose coupling



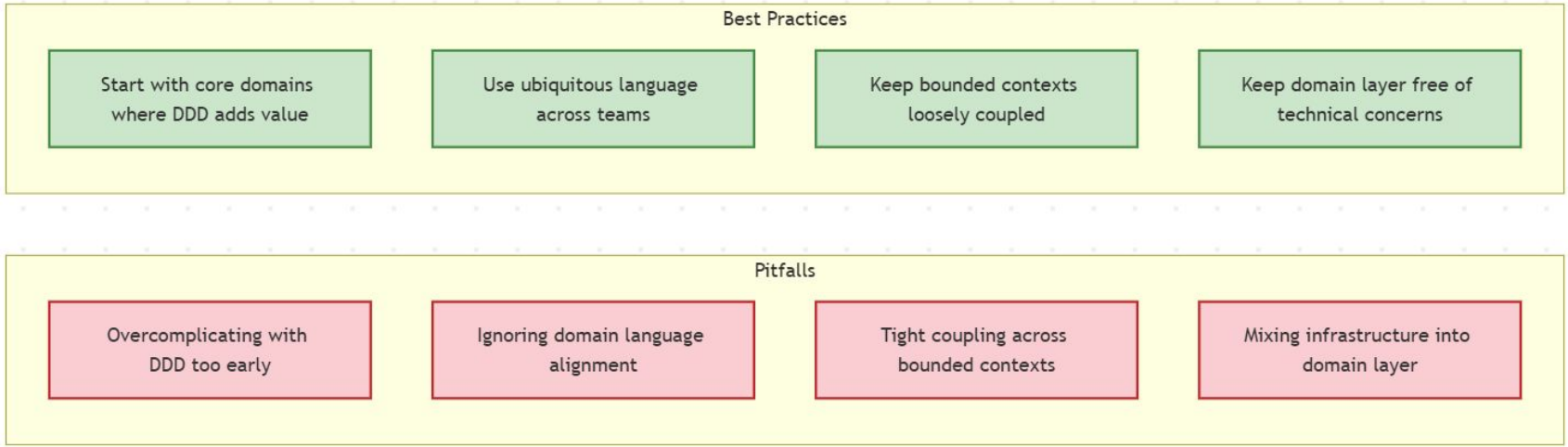
Organizational Impact – Aligning Teams & Architecture

- Teams structured around bounded contexts or subdomains
- Improves communication and reduces cognitive load
- Encourages ownership and faster feedback loops
- Aligns technical architecture with organizational structure (Conway's Law)



Common Challenges & Best Practices

- Avoid overcomplicating with premature DDD
- Invest in domain exploration and ubiquitous language
- Keep bounded contexts loosely coupled but consistent internally
- Use tooling and automation to enforce boundaries and contracts



Summary – DDD as a Foundation for Masterful Architecture

- DDD bridges domain knowledge and system design
- Bounded contexts and aggregates shape modular, scalable architecture
- Supports layered, maintainable, and testable systems
- Empowers teams and enables strategic integration
- What's next:
  - Conclusion & Way Forward